

Lecture 11 - DSA

13 May 2026

1) Cuckoo hashing

- 2 hash tables

h_1 T_1

h_2 T_2

2 hash tble of size 11: ($m=11$)

	0	1	2	3	4	5	6	7	8	9	10
T_1	1	100		3			50	1	20		

	0	1	2	3	4	5	6	7	8	9	10
T_2	3	20		39	53		75			100	

$$h_1(k) = k \% 11$$

$$h_2(k) = (k \text{ div } 11) \% 11$$

k	h_1	h_2
20	9	1
50	6	4
	$(50 \% 11)$	$(50 / 11) \% 11$
53	9	4

eliminate 20 → put 53 instead. → 20 to T_2

75 9 6 → 53 to T_2

100 1 9

67 1 6

↳ occupied ⇒ move 100 to T_2 → put 67 instead.

105 6 9 → 50 to T_2

3 3 0 → 53 to T_1

↳ available

36 3 3 → 75 to T_2

↳ occupied ⇒ 3 to T_2 on pos 0 (available

6 3

105 to T_2 on pos

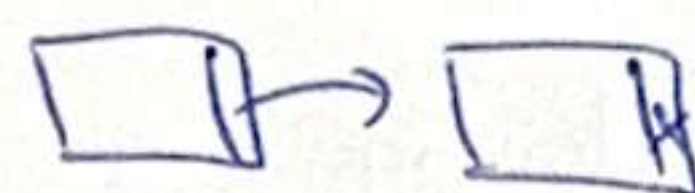
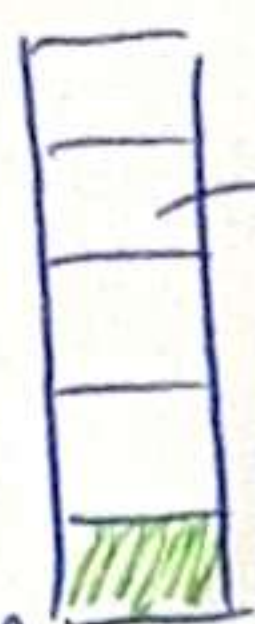
100 to T_1

111
600 ← 39

2) Perfect hashing $\rightarrow$ no collisions, but you have to know all the elements that are going to be added in advance. $\Theta(1)$

$\rightarrow$ separate chaining $\alpha = \frac{m}{m}$
how much to increase m ?

If $m = m^2 \Rightarrow 50\%$ chance there are no collisions in the hash tbl.



$m =$ table size

$m = ?$ nr of elements

= hash table of hash tables

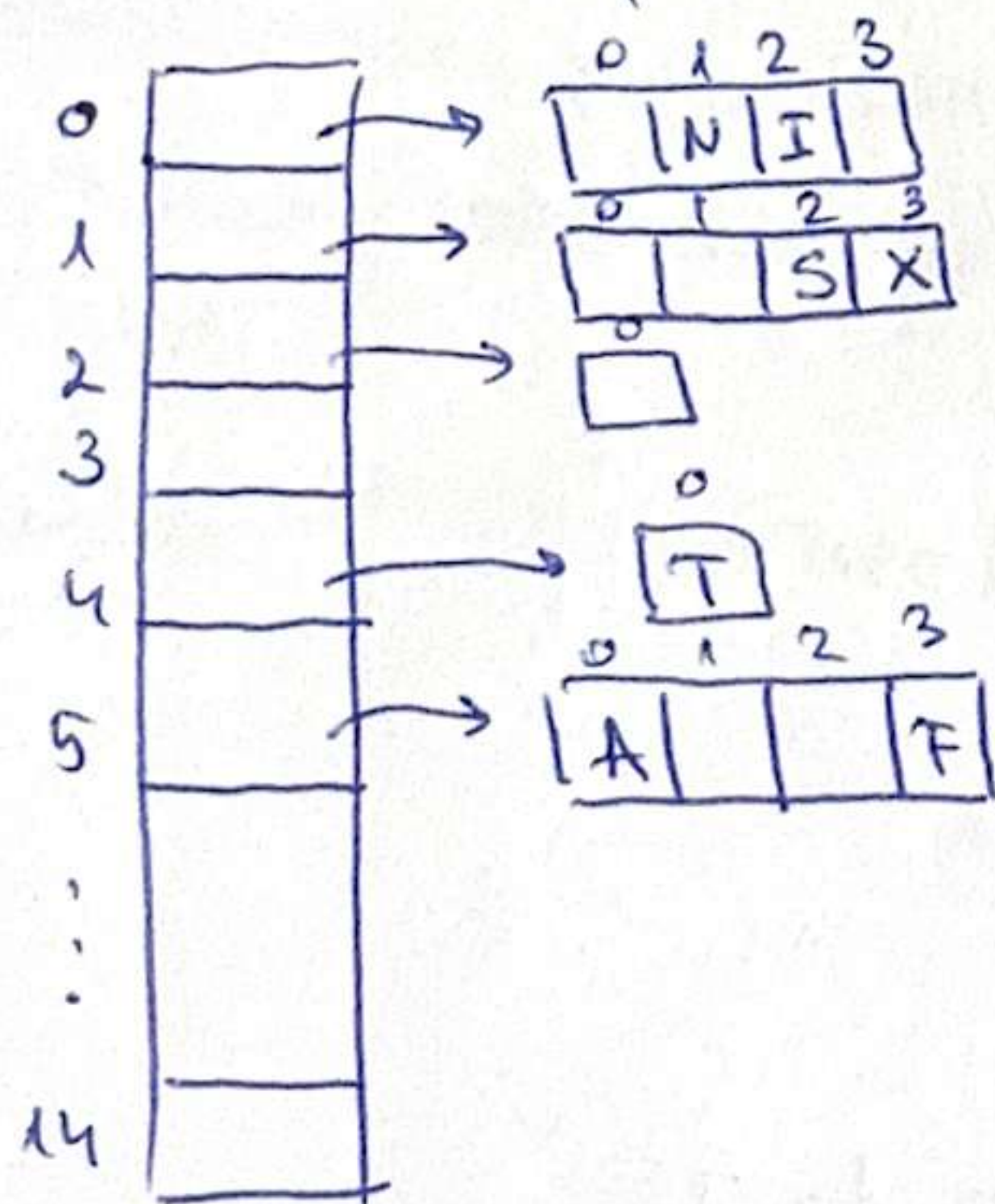
$\rightarrow$ only the hash functions that hash to that position.

$\rightarrow$ universal hashing $\rightarrow$ params uniquely initialised s.t. each time you get a $\neq$ hash function.

$$H = \{H_{a,b}(x) = ((a * x \dots))\}$$

• Insert in hash table w perf. hashing the unique letters from "..."
 $\rightarrow$ consider only unique letters.

$a=3, b=2$ (random) $\Rightarrow$ one hash function $\Rightarrow$ positions for elems
each position may have $>$ letters.



$a=4, b=11$
another hash table whose size is $m^2 = b^2$?

~~$a=5, b=2$~~ $a=2, b=13$

$h(k)=0$ only one letter $\rightarrow E$.

$h(k)=0$

$$a=3, b=5 \quad ((3x+5)\%29)\%4$$

F has hash code 6 $\Rightarrow ((3*6+5)\%29)\%4 = 3$ (pos)

A has hash code 1 $\Rightarrow ((3+5)\%29)\%4 = 0$ (pos)

Searching = compute hash ^{table} value of 3 values. $\Theta(1)$

remove = search + (empty)

insertion = ...

3) Linked hash table - Java

$\rightarrow$ separate-chaining based. $\rightarrow$ don't know the printing order.
(depends on size $m =$ table size)

Node:

info: TKey

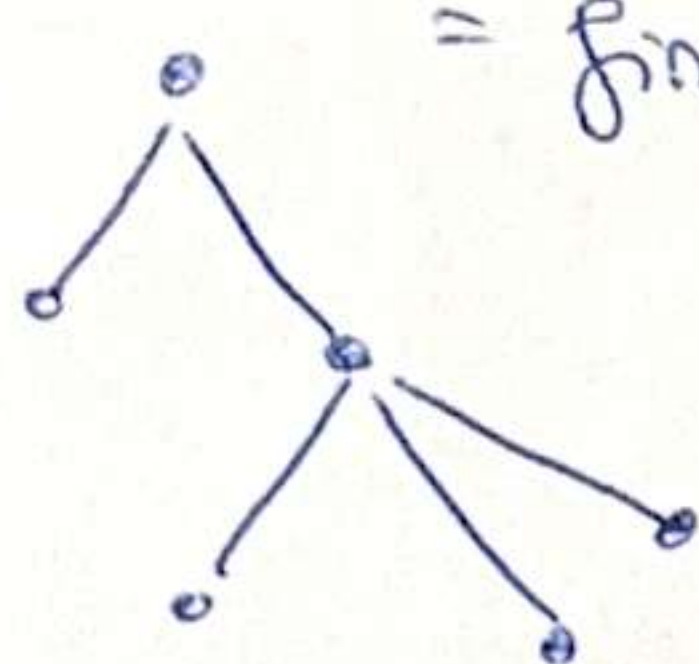
nextH: $\uparrow$ Node

nextL: $\uparrow$ Node

prevL: $\uparrow$ Node

Trees = connected, acyclic graph

= finite set of 0 or more elements = NODES



→ each child is a tree itself.

→ degree node defined by nr. of children

→ leaf (no children)

root

• Depth of a node → from the root, count nr of EDGES. (unique)

• Height — — — — — → from leaf towards one node (not unique)
↳ the LONGEST ONE.

leaf

• Height of tree = height to the ROOT.

I. K-ary tree (max nr. of children = K)

Tree traversal = go through all the nodes ~ search

↳ Depth first traversal: start from root → go ↓ from root.
↳ stack

stack: R, ~~X~~, ~~Y~~, ~~Z~~, ~~K~~, ~~P~~, ~~H~~, ~~Q~~, ~~Y~~

visited: R B T A G J Z K O H Y Q P

→ push from right to left children on the stack

→ if from left to right, still DFS, but start from right side

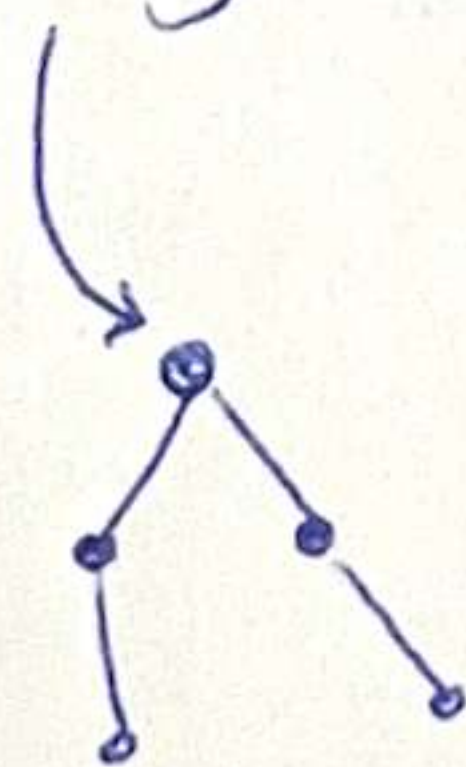
↳ Level order traversal: start from root → go ↓ from root
↳ queue

queue: ~~R~~, ~~B~~, ~~G~~, ~~J~~, ~~T~~, ~~A~~, ~~Z~~, ~~K~~, ~~O~~, ~~H~~, ~~Q~~, ~~P~~, ~~Y~~

visited: R B G J T A Z K O H Q P Y

II. Binary trees (max 2 children / node)

left child right child → ordered.



= FULL ⇒ exactly 2 children for every internal node
(≠ COMPLETE) → (nr children 0 or 2)

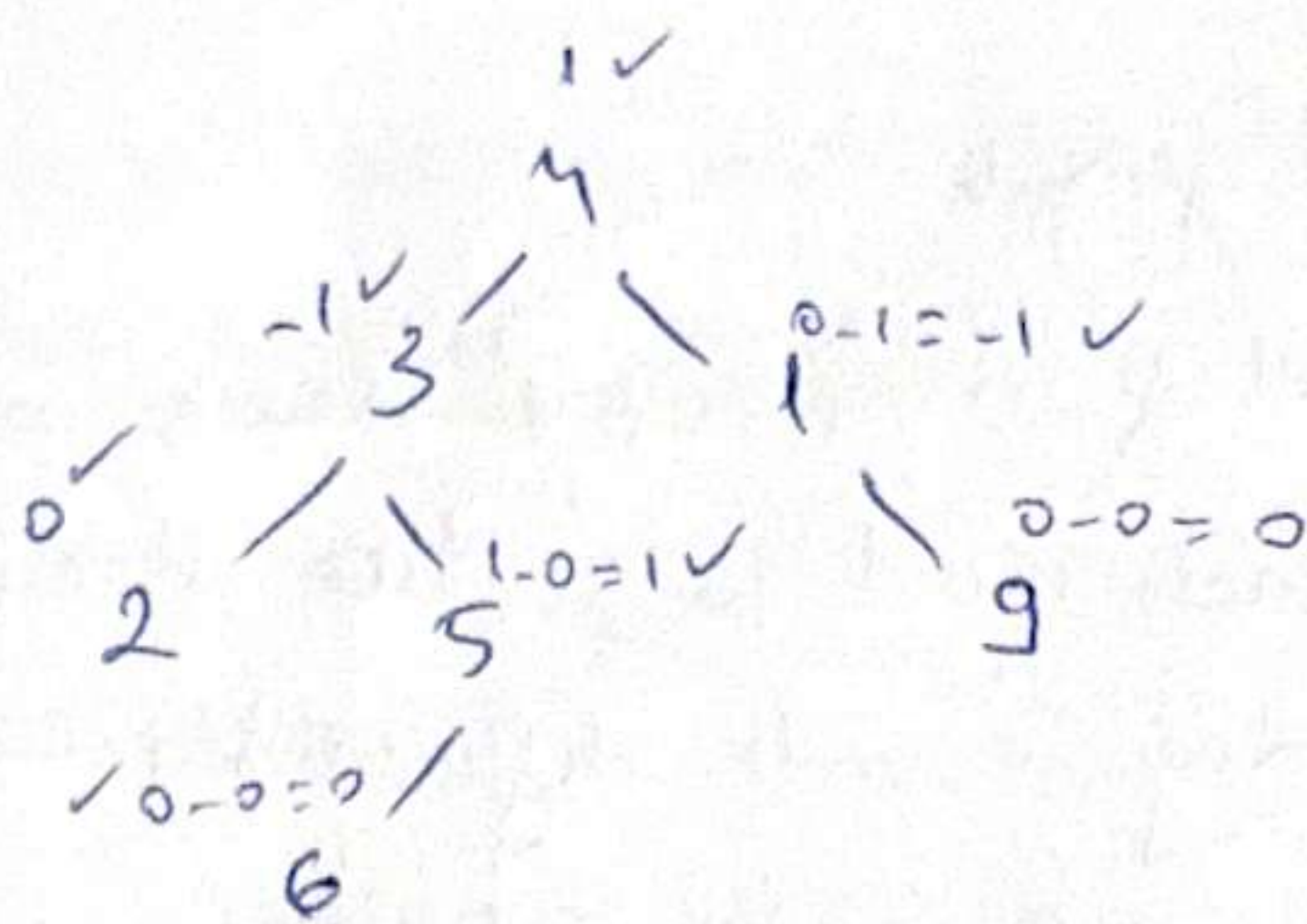
= COMPLETE ⇒ can't add any more nodes without increasing the height.

= ALMOST COMPLETE ⇒ heap structure

= DEGENERATE ⇒ chain

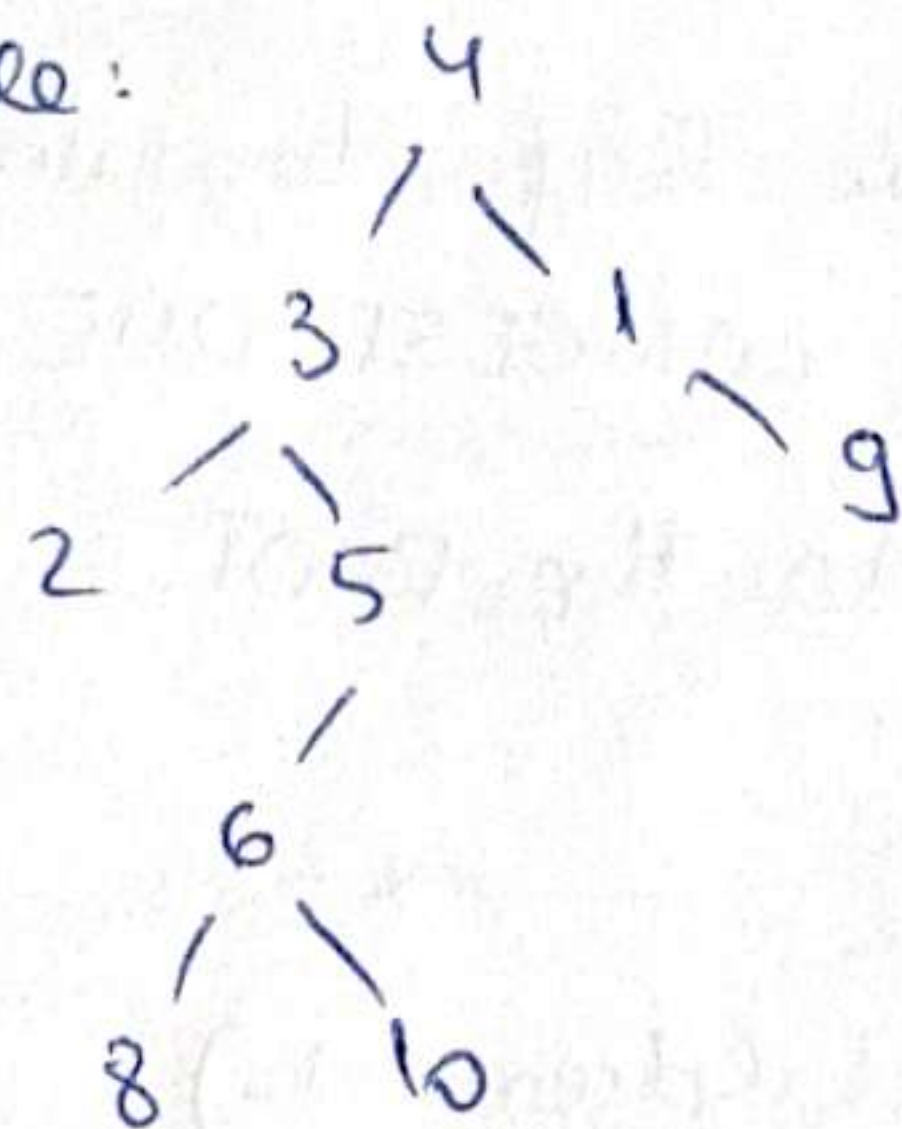
= BALANCED ⇒ $\text{height}(\text{left}) - \text{height}(\text{right}) \leq 1$





• Where to add to keep balanced $\Rightarrow$ near 9.

"random" tree example:



not balanced
not degenerate
not complete
⋮

Properties:

- 1) m nodes $\Rightarrow m-1$ edges
- 2) complete $\Rightarrow 2^{N+1} - 1$ nodes = max nr. nodes.
- 3) ?
- 4) min $\rightarrow N+1$ (degenerate)
- 5) $N-1$ (complete)

ADT Binary Tree $\rightarrow$ container.

- remove - does not make sense
- add

$\rightarrow$ create $\checkmark$ `initTree (bt, left, e, right)`